

Experiment 10 – Transactions and Savepoints

Sahil Goyal | UID: 24BDA70148

Aim

To study transactions, rollback, savepoints and the ACID properties in PostgreSQL, and to build a transaction that inserts several student records in one go while isolating any failed insert.

Objective

- Insert multiple rows inside a single transaction.
- Roll back only the failed insert while keeping earlier successful inserts (savepoint behaviour).
- Print clear success and failure messages for every insert.
- Preserve data integrity using constraints (UNIQUE, CHECK).

Question

Design a PostgreSQL transaction for the students table in which multiple student records are inserted together. If an insert fails because of invalid data, only that insert should be rolled back and the earlier successful inserts must be preserved, using savepoints. The system should report a clear message for each successful and each failed insertion, ensuring data integrity and controlled error handling.

Schema

```
DROP TABLE IF EXISTS students;

CREATE TABLE students (
    id    SERIAL PRIMARY KEY,
    name  VARCHAR(50) NOT NULL UNIQUE,
    age   INT CHECK (age > 0),
    class INT NOT NULL
);
```

Query

The worksheet's version re-raised the error, which rolled back the whole block and lost the earlier inserts — the opposite of what the question asks. In PL/pgSQL, a nested BEGIN ... EXCEPTION block acts as an automatic savepoint, so each insert gets its own block and only the failing one is undone.

```
DO $$
BEGIN
    -- insert 1
    BEGIN
        INSERT INTO students(name, age, class) VALUES ('Anisha', 16, 8);
        RAISE NOTICE 'Insert 1 OK: Anisha';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'Insert 1 FAILED (rolled back): %', SQLERRM;
    END;

    -- insert 2
    BEGIN
        INSERT INTO students(name, age, class) VALUES ('Mayank', 19, 9);
        RAISE NOTICE 'Insert 2 OK: Mayank';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'Insert 2 FAILED (rolled back): %', SQLERRM;
    END;

    -- insert 3: duplicate name, violates UNIQUE
    BEGIN
```

```

        INSERT INTO students(name, age, class) VALUES ('Anisha', 17, 8);
        RAISE NOTICE 'Insert 3 OK: Anisha';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'Insert 3 FAILED (rolled back): %', SQLERRM;
    END;

    -- insert 4
    BEGIN
        INSERT INTO students(name, age, class) VALUES ('Rohan', 18, 10);
        RAISE NOTICE 'Insert 4 OK: Rohan';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'Insert 4 FAILED (rolled back): %', SQLERRM;
    END;

    -- insert 5: negative age, violates CHECK
    BEGIN
        INSERT INTO students(name, age, class) VALUES ('Kabir', -5, 9);
        RAISE NOTICE 'Insert 5 OK: Kabir';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'Insert 5 FAILED (rolled back): %', SQLERRM;
    END;

    RAISE NOTICE 'Transaction completed: successful inserts were kept.';
END;
$$;

SELECT * FROM students ORDER BY id;

```

Output

Unsaved e

```

2 BEGIN
35 -- insert 5: negative age, violates CHECK
36 BEGIN
37     INSERT INTO students(name, age, class) VALUES ('Kabir', -5, 9);
38     RAISE NOTICE 'Insert 5 OK: Kabir';
39 EXCEPTION WHEN OTHERS THEN
40     RAISE NOTICE 'Insert 5 FAILED (rolled back): %', SQLERRM;
41 END;
42
43 RAISE NOTICE 'Transaction completed: successful inserts were kept.';
44 END;
45 $$;
46
47 SELECT * FROM students ORDER BY id;
48

```

Results Chart

id	name	age	class
1	Anisha	16	8
2	Mayank	19	9
4	Rohan	18	10

Learning Outcome

1. Understood and implemented transactions, rollback and savepoint behaviour in PostgreSQL.
2. Used PL/pgSQL exception handling so a failed statement is rolled back without discarding earlier work.
3. Used constraints and clear notices to keep data consistent and errors controlled.